

Clase 13

ABSTRACCIÓN FUNCIONAL (PRIMERA PARTE)

CC1002 Introducción a la
Programación

Similitudes en estructuras y funciones

- Hay estructuras de datos que son parecidas
 - Ejemplo: lista de enteros y lista de Strings
- Funciones asociadas a estas estructuras también pueden ser similares
 - Dos funciones muy parecidas: se repite código
- ¿Cómo programar funciones genéricas que eviten la repetición de código?

CC1002 Introducción a la
Programación

Ejemplo

- Compare los siguientes códigos:

```
# hayPelotas: lista(str) -> bool
# determina si unaLista contiene
# el string 'pelota'
# Ejemplo: hayPelotas(lista('pelota', listaVacía))
# devuelve True
def hayPelotas(unaLista):
    if vacía(unaLista):
        return False
    else:
        if cabeza(unaLista) == 'pelota':
            return True
        else:
            return hayPelotas(cola(unaLista))

# hayAutos: lista(str) -> bool
# determina si unaLista contiene
# el string 'auto'
# Ejemplo: hayAutos(lista('auto', listaVacía))
# devuelve True
def hayAutos(unaLista):
    if vacía(unaLista):
        return False
    else:
        if cabeza(unaLista) == 'auto':
            return True
        else:
            return hayAutos(cola(unaLista))
```

CC1002 Introducción a la
Programación

Ejemplo

- Compare los siguientes códigos:

```
# hayPelotas: lista(str) -> bool
# determina si unaLista contiene
# el string 'pelota'
# Ejemplo: hayPelotas(lista('pelota', listaVacía))
# devuelve True
def hayPelotas(unaLista):
    if vacía(unaLista):
        return False
    else:
        if cabeza(unaLista) == 'pelota':
            return True
        else:
            return hayPelotas(cola(unaLista))

# hayAutos: lista(str) -> bool
# determina si unaLista contiene
# el string 'auto'
# Ejemplo: hayAutos(lista('auto', listaVacía))
# devuelve True
def hayAutos(unaLista):
    if vacía(unaLista):
        return False
    else:
        if cabeza(unaLista) == 'auto':
            return True
        else:
            return hayAutos(cola(unaLista))
```

CC1002 Introducción a la
Programación

Ejemplo

- De la comparación resalta:
 - Ambas funciones consumen una lista de Strings
 - Ambas funciones buscan un String en particular ('pelota' y 'auto', respectivamente)
 - Aparte del String distinto, ambas funciones son indistinguibles
 - ¿Y si ahora se necesita buscar por 'avion'?

CC1002 Introducción a la
Programación

Ejemplo

- Función más general: busca cualquier string

```
# contiene: str lista(str) -> bool
# Determina si unaLista contiene el String valor
# ejemplo: contiene('auto', lista('auto', listaVacia)) retorna True
def contiene(valor, unaLista):
    if vacia(unaLista):
        return False
    else:
        if cabeza(unaLista) == valor:
            return True
        else:
            return contiene(valor, cola(unaLista))

# Tests
assert contiene('auto', lista('auto', listaVacia))
```

- Diferencia: parámetro extra **valor**

CC1002 Introducción a la
Programación

Abstracción funcional

- Al proceso de combinar dos o más funciones en una sola más genérica se le denomina **abstracción funcional**
- La abstracción funcional es muy beneficiosa en programación
- Requiere agregar/modificar parámetros a las funciones para hacerlas genéricas

CC1002 Introducción a la
Programación

Abstracción funcional

- Compare las siguientes funciones:

```
# inferiores: lista(num) num -> lista(num)
# devuelve una lista con aquellos numeros
# de unaLista que sean inferiores a n
# Ejemplo: inferiores(lista(1, lista(2, listaVacia)), 2)
# devuelve lista(1, listaVacia)
def inferiores(unalista, n):
    if vacia(unalista):
        return listaVacia
    else:
        if cabeza(unalista) < n:
            return lista(cabeza(unalista), inferiores(cola(unalista), n))
        else:
            return inferiores(cola(unalista), n)

# superiores: lista(num) num -> lista(num)
# devuelve una lista con aquellos numeros
# de unaLista que sean superiores a n
# Ejemplo: superiores(lista(1, lista(2, listaVacia)), 1)
# devuelve lista(2, listaVacia)
def superiores(unalista, n):
    if vacia(unalista):
        return listaVacia
    else:
        if cabeza(unalista) > n:
            return lista(cabeza(unalista), superiores(cola(unalista), n))
        else:
            return superiores(cola(unalista), n)
```

CC1002 Introducción a la
Programación

Abstracción funcional

- Compare las siguientes funciones:

```
# inferiores: lista(num) num -> lista(num)
# devuelve una lista con aquellos numeros
# de una lista que sean inferiores a n
# Ejemplo: inferiores(lista(1, lista(2, listaVacía)), 2)
# devuelve lista(1, listaVacía)
def inferiores(unalista, n):
    if vacía(unalista):
        return listaVacía
    else:
        if cabeza(unalista) < n:
            return lista(cabeza(unalista), inferiores(cola(unalista), n))
        else:
            return inferiores(cola(unalista), n)

# superiores: lista(num) num -> lista(num)
# devuelve una lista con aquellos numeros
# de una lista que sean superiores a n
# Ejemplo: superiores(lista(1, lista(2, listaVacía)), 1)
# devuelve lista(2, listaVacía)
def superiores(unalista, n):
    if vacía(unalista):
        return listaVacía
    else:
        if cabeza(unalista) > n:
            return lista(cabeza(unalista), superiores(cola(unalista), n))
        else:
            return superiores(cola(unalista), n)
```

CC1002 Introducción a la
Programación

Abstracción funcional

- La diferencia es el operador de comparación
 - Definir una función para el operador, y agregar un parámetro extra
- En Python se puede recibir funciones como argumento de una función
 - Se denominan *high-order functions*
- Función **filtro**: filtra algunos de los elementos de la lista

CC1002 Introducción a la
Programación

Abstracción funcional

- Definir función para el operador:

```
# menorQue: num num -> bool
# devuelve True si el primer argumento es menor que el segundo
# y False en el caso contrario
# Ejemplo: menorQue(4,2) devuelve False
def menorQue(x,y):
    return x < y

# Tests
assert not menorQue(4, 2)
```

- La función **mayorQue** se puede definir en forma similar

CC1002 Introducción a la
Programación

Abstracción funcional

- Realizando la abstracción funcional:

```
# filtro: (num num -> bool) lista(num) num -> lista(num)
# devuelve una lista con todos los valores donde la funcion operador es True
# para el valor n dado
# ejemplo: ...
def filtro(operador, unaLista, n):
    if vacia(unaLista):
        return listaVacia
    else:
        if operador(cabeza(unaLista), n):
            return lista(cabeza(unaLista), filtro(operador, cola(unaLista), n))
        else:
            return filtro(operador, cola(unaLista), n)

# Tests
valores = lista(6, lista(4, lista(8, listaVacia)))
assert filtro(menorQue, valores, 5) == lista(4, listaVacia)
```

CC1002 Introducción a la
Programación

Abstracción funcional

- Función **filtro** genérica:

```
# filtro: (X -> bool) lista(X) -> lista(X)
# devuelve una lista con todos los valores donde la funcion f devuelve True
# ejemplo: ...
def filtro(f, unaLista):
    if vacia(unaLista):
        return listaVacia
    else:
        if f(cabeza(unaLista)):
            return lista(cabeza(unaLista), filtro(f, cola(unaLista)))
        else:
            return filtro(f, cola(unaLista))
```

CC1002 Introducción a la
Programación

Abstracción funcional

- Ejemplo de uso función **filtro**:

```
# esNegativo: num -> bool
# devuelve True si el numero x es negativo, False en el caso contrario
# Ejemplo: esNegativo(4) devuelve False
def esNegativo(x):
    return x < 0

>>> valores = lista(5, lista(8, lista(-4, lista(6, lista(-7, lista(2, listaVacia))))))
>>> print(filtro(esNegativo, valores))
lista(valor=-4, siguiente=lista(valor=-7, siguiente=None))
```

CC1002 Introducción a la
Programación

Abstracción funcional

- Otros tipo de problemas con listas:
 - Sumarle uno a todos los valores de la lista
 - Calcular la raíz cuadrada a todos los valores de la lista
 - Convertir una lista con temperaturas de °C a °F
- ¿Qué tienen en común todos estos problemas?

CC1002 Introducción a la
Programación

Abstracción funcional

- Todos esos problemas implican aplicar una función a cada elemento de la lista
 - Función **mapa**: recibe una lista y una función, y devuelve una lista con la función aplicada a todos los elementos de la lista

CC1002 Introducción a la
Programación

Abstracción funcional

- Función **mapa**:

```
# mapa: (X -> Y) lista(X) -> lista(Y)
# devuelve lista con funcion aplicada a todos sus elementos
# ejemplo: ...
def mapa(f, unaLista):
    if vacia(unaLista):
        return listaVacia
    else:
        return lista(f(cabeza(unaLista)), mapa(f, cola(unaLista)))
```

CC1002 Introducción a la
Programación

Abstracción funcional

- Ejemplo de uso función **mapa**:

```
# porDiez: num -> num
# devuelve la multiplicacion de numero por 10
def porDiez(numero):
    return 10 * numero

>>> valores = lista(1, lista(2, lista(3, lista(4, listaVacia))))
>>> print(mapa(porDiez, valores))
lista(valor=10, siguiente=lista(valor=20, siguiente=lista(valor=30,
siguiente=lista(valor=40, siguiente=None))))
```

CC1002 Introducción a la
Programación

Abstracción funcional

- Usando una función anónima:

```
>>> valores = lista(1, lista(2, lista(3, lista(4, listaVacía))))  
>>> print(mapa(lambda x: 10 * x, valores))  
lista(valor=10, siguiente=lista(valor=20, siguiente=lista(valor=30,  
siguiente=lista(valor=40, siguiente=None))))
```

CC1002 Introducción a la
Programación

Para la próxima clase

- Leer los Capítulos 10.3 a 10.6 del apunte

CC1002 Introducción a la
Programación